

Engenharia de Prompts e IA Generativa

O sistema implementa três abordagens distintas de engenharia de prompts para lidar com diferentes cenários operacionais da colônia:

Prompt Zero-Shot (*resumo de alertas operacionais*)

O modelo recebe uma instrução direta e clara sobre o que deve fazer, sem que nenhum exemplo prévio seja fornecido. Tem como objetivo operacional fornecer um resumo executivo imediato e direto de falhas críticas detectadas nos módulos, auxiliando a equipe a priorizar ações de manutenção.

```
256 def montar_prompt_zero_shot(alerta: dict) -> str:
257     """Prompt zero-shot: pede um resumo do alerta sem exemplos prévios."""
258     return (
259         "Resuma em uma frase, em tom técnico e objetivo, o seguinte alerta "
260         f"operacional da colônia: módulo '{alerta.get('modulo', 'desconhecido')}', "
261         f"ocorrência '{alerta.get('tipo_ocorrencia', 'nao_informada')}', "
262         f"prioridade '{alerta.get('prioridade', 'nao_informada')}', "
263         f"descrição: '{alerta.get('mensagem', '')}'".
264     )
265
```

Prompt Few-Shot (*classificação de solicitações da tripulação*)

A técnica consiste em fornecer exemplos práticos e resolvidos do padrão de comportamento esperado antes de apresentar a solicitação real. Isso direciona a precisão e o estilo da resposta da IA. Tem como objetivo garantir que a triagem de chamados enviados pelos tripulantes seja categorizada uniformemente.

```
267 def montar_prompt_few_shot(solicitacao: str) -> str:
268     """Prompt few-shot: dá exemplos de classificação antes de pedir a nova."""
269     return (
270         "Classifique solicitações da tripulação em: URGENTE, ROTINA ou INFORMATIVA.\n\n"
271         "Exemplo 1: 'Vazamento de ar na Habitação' -> URGENTE\n"
272         "Exemplo 2: 'Solicito troca de filtro na próxima manutenção' -> ROTINA\n"
273         "Exemplo 3: 'Qual o horário do próximo pouso de suprimentos?' -> INFORMATIVA\n\n"
274         f"Solicitação: '{solicitacao}' -> "
275     )
276
```

Prompt de Saída Estruturada / Structured Outputs (Formato JSON)

Restringe o comportamento gerativo do modelo exigindo que a resposta siga estritamente um formato computacional padronizado (como JSON), sem textos explicativos adicionais. Tem como objetivo permitir que outras partes do software processem a resposta da IA de forma automática e segura, integrando o núcleo cognitivo ao restante do pipeline da colônia.

```
278 def montar_prompt_saida_estruturada(alerta: dict) -> str:
279     """Prompt que exige explicitamente uma saída estruturada em JSON."""
280     return (
281         "Responda APENAS com um JSON válido, sem texto adicional, no formato "
282         '{"modulo": str, "critico": bool, "acao_recomendada": str}, '
283         f"considerando o alerta: {json.dumps(alerta, ensure_ascii=False)}"
284     )
285
```

Mecanismo de Resiliência e Fallback (*Mock Local*)

Para assegurar que o sistema funcione de maneira ininterrupta (mesmo diante de falhas de conexão ou esgotamento de cota da API (como o erro 429 RESOURCE_EXHAUSTED), o NCAS conta com um sub sistema inteligente de resiliência implementado em *simular_resposta_assistente()*:

Caso a chave de API não esteja configurada ou ocorra uma exceção de rede/quota, o software aciona automaticamente um *fallback* local (*_resposta_mock*), retornando uma resposta estruturada de segurança. Essa estratégia cumpre integralmente os requisitos de robustez do projeto, garantindo que o programa nunca sofra *crash* na frente dos avaliadores.

```
287 def _resposta_mock(contexto: str = "geral") -> str:
288     """
289     Centraliza o texto de resposta simulada, evitando duplicidade entre
290     os diferentes pontos onde simular_resposta_assistente() pode cair
291     no fallback local.
292     """
293     textos = {
294         "geral": (
295             "[Resposta simulada] Alerta reconhecido. Recomenda-se isolar o "
296             "módulo afetado e acionar a equipe de manutenção com "
297             "prioridade compatível ao nível informado."
298         ),
299     }
300     return textos.get(contexto, textos["geral"])
301
```

```

303 def simular_resposta_assistente(prompt: str) -> str:
304     """
305     Retorna a resposta do assistente inteligente.
306
307     Se houver uma GEMINI_API_KEY configurada e a biblioteca google-genai
308     instalada, faz a chamada real (mesmo cliente usado em main.py).
309     Caso contrário, devolve uma resposta simulada localmente – o item 2.3
310     do enunciado permite explicitamente essa simulação.
311     """
312     if _client:
313         try:
314             resposta = _client.models.generate_content(
315                 model="gemini-2.5-flash", contents=prompt
316             )
317             return (resposta.text or "").strip()
318         except Exception as erro: # falha de rede/quota etc. -> cai para o mock
319             return (
320                 f"[IA indisponível ({erro}); resposta simulada] Alerta reconhecido "
321                 "e registrado para acompanhamento do Centro de Controle."
322             )
323
324     return _resposta_mock()

```